

Performance Tuning — DocumentIA

Última actualización: 2026-06-10
Aplicable a: Azure Functions v4, Durable Functions, .NET 10 Isolated

1. Benchmarks & Expectativas

Este documento establece benchmarks de rendimiento realistas y proporciona guías de tuning para los tres ambientes principales: Local (Dev), Development (Staging) y Production.

1.1 Ambiente Local (Dev)

Métrica	Valor	Notas
Tiempo clasificación single doc	2-5 sec	Con Azurite local, sin latencia de red
Throughput	1 doc/sec (single instance)	Sequential execution, single core
Recursos esperados	2GB RAM, 2 cores	Laptop dev típico
P95 latency	< 6 sec	Ambiente controlado
Disponibilidad	100%	Ambiente no productivo
Error rate	< 1%	Excepto test de fallos inducidos

Escenario típico:
1 documento notasimple + Azure Content Understanding (mock local) → ~2-3 sec total

1.2 Ambiente Development (Staging)

Métrica	Valor	Notas
Tiempo clasificación single doc	3-8 sec	Con CU real + fallback GPT
Throughput	10 docs/sec (scaled instance, 4 concurrent activities)	Multidoc concurrente
P50 latency	< 5 sec	Mediana típica
P95 latency	< 15 sec	Percentil 95
P99 latency	< 25 sec	Percentil 99
Availability	99.5%	SLA objetivo staging
Error rate	< 2%	Incluye circuit breaker trips, retries
Max concurrent docs	40+	10 instances × 4 concurrent activities

Desglose típico de latencia (8 sec total):

- NormalizarActivity: 100ms
- SubirBlobActivity: 200ms
- ClasificarActivity (CU hybrid + rules): 2-3 sec
- ExtraerActivity (CU + DI fallback): 2-3 sec
- IntegrarActivity + ValidarActivity: 500ms
- SubirGDCActivity (SOAP): 1-2 sec
- PersistirActivity: 200ms

1.3 Ambiente Production

Métrica	Valor	Notas
Tiempo clasificación single doc	5-12 sec	Con todos los fallbacks activos
Throughput baseline	100-500 docs/day	Carga típica operativa
Throughput peak	1000+ docs/day	Con autoscale habilitado
P50 latency	< 8 sec	Mediana con latencia de red
P95 latency	< 30 sec	Percentil 95 con variabilidad
P99 latency	< 45 sec	Percentil 99 con outliers ocasionales
Availability SLA	99.9%	Requisito contractual
Error rate	< 1%	Excluye reintentos exitosos
Cost per document	\$0.02-0.05	Incluyendo CU, OpenAI, SQL, Storage
Max concurrent docs	100+	20 instances × 4 concurrent + autoscale

Desglose típico de latencia (10 sec total):

- Normalización + Deduplicación: 300ms
- Subir a Blob: 500ms
- Clasificación (todos fallbacks): 3-4 sec
- Extracción (CU + DI + GPT): 3-4 sec
- Validación + Integración: 800ms
- Subir a GDC (SOAP con reintentos): 1.5-2 sec
- Persistencia: 300ms

Cost Breakdown por componente (USD):

Componente	Costo/doc	Notas
Azure CU	\$0.002-0.005	2-5 páginas × \$0.0001-0.002/pág
OpenAI (fallback)	\$0.01-0.05	150 tokens prompt + response
SQL DB (queries)	\$0.0001-0.001	~5 queries/doc, DTU variable
Storage	negligible	~1 blob/doc, read/write minimal
Functions (compute)	\$0.0002-0.001	Depends on cores y duración
Total estimado	\$0.02-0.06	Incluye picos y reintentos

2. Componentes & Tuning Points

2.1 Azure Functions / Durable Orchestrator

Ubicación: `host.json` , `appsettings.json`

Configuración actual (default):

```
{
  "extensions": {
    "durableTask": {
      "hubName": "DocumentIAHub",
      "maxConcurrentActivityFunctions": 4,
      "maxConcurrentOrchestratorFunctions": 4,
      "extendedSessionsEnabled": false
    }
  }
}
```

Tuning points:

Parámetro	Default	Rango recomendado	Impacto
<code>maxConcurrentActivityFunctions</code>	4	4-12 (scaling)	Paralelismo de actividades. Aumentar para I/O-bound (Cloud API calls), no para CPU-bound.
<code>maxConcurrentOrchestratorFunctions</code>	4	2-4 (mantener bajo)	Ejecución de lógica del orquestador. No afecta paralelismo de actividades.
<code>extendedSessionsEnabled</code>	false	true (si > 10 docs/sec)	Mantiene sesión del orquestador en memoria. Reduce latencia, aumenta RAM.

Recomendaciones:

- **Local Dev:** Dejar como default (4, 4, false)
- **Staging:** Aumentar a (8, 4, false) para test de carga
- **Production:** Usar (12, 4, false) o (8, 4, true) según carga y latencia crítica

Impacto esperado:

- `maxConcurrentActivityFunctions` 4→8: +50% throughput, ~100ms latencia media
- `extendedSessionsEnabled` false→true: -100-200ms latencia, +5-10% RAM

2.2 Activity Functions (por tipo)

El orquestador ejecuta 14+ actividades en secuencia/paralelo. Cada una tiene características y timeouts específicos.

Actividades I/O-heavy (candidatas a paralelismo):

Actividad	Tipo	Timeout	Características	Tuning
IngestActivity	I/O	5m	Lectura de blob	Aumentar timeout si doc > 100MB
SubirBlobActivity	I/O	2m	Upload a Storage	Pool size Storage Account
VerificarDuplicadoActivity	DB Query	30s	SHA256 lookup	Index verificado, típicamente < 50ms
ObtenerUltimaEjecucionActivity	DB Query	30s	Última clasificación	Index, típicamente < 100ms
ClasificarActivity	API Call	60s	CU hybrid + rules + GPT fallback	Aumentar timeout si fallback activo
ExtraerActivity	API Call	90s	CU/DI + GPT fallback	Circuit breaker habilitado (CU)
ExtraerMarkdownLayoutActivity	API Call	60s	DI layout extraction	Timeout típico: 2-5 sec
PromptActivity	API Call	60s	GPT prompt libre	Típico: 3-10 sec
ValidarActivity	CPU	30s	Validación reglas + regex	CPU-bound, típico: 100-500ms
IntegrarActivity	API Call	120s	Plugin manager (REST, SOAP, DLL)	Depende de plugins, típico: 2-5 sec
ObtenerActivoActivity	DB Query	60s	Asset resolver (Activo search)	DB query + búsqueda, típico: 1-3 sec
SubirGDCActivity	API Call	120s	SOAP GDC upload	Timeout habitual: 5-10 sec
PersistirActivity	DB Write	30s	EF Core save + audit	Típico: 200-500ms

Timeouts actuales en código (ejemplos):

- Classification GPT: `TimeoutSeconds: 30` (appsettings.json)
- Extraction CU: `HardTimeoutSeconds: 90` (appsettings.json)
- GDC Upload: implícito ~120s (DocumentProcessOrchestrator context)

Tuning recomendado:

- **Reducir latencia:** Identificar actividades lentas (ver AppInsights), aumentar su timeout solo si es network-bound
- **Aumentar throughput:** Parallelizar actividades independientes mediante `Task.WhenAll()` en orquestador si es posible
- **Fallos transitorios:** Aumentar `maxRetries` en providers con alta variabilidad (CU, GPT)

2.3 External Plugins / Providers

2.3.1 Azure Content Understanding (CU)

Configuración actual:

```
{
  "AzureContentUnderstanding": {
    "MaxConcurrentCalls": 4,
    "HardTimeoutSeconds": 90,
    "EnableCircuitBreaker": true,
    "CircuitBreakerFailureThreshold": 5,
    "CircuitBreakerOpenSeconds": 45,
    "MaxRetries": 3,
    "InitialRetryDelayMs": 500
  }
}
```

Límites conocidos (by tier):

- Rate limit: 100 req/min (Tier: Standard; puede variar)
- Max document size: 20MB (típico)
- Timeout default: 90 sec (configurable)
- Pages limit: 500 páginas/doc (típico)

Tuning:

Parámetro	Default	Rango	Impacto
MaxConcurrentCalls	4	4-10	Paralelismo local. Aumentar si carga > 10 docs/sec.
HardTimeoutSeconds	90	60-120	Timeout total. Aumentar si docs complejos frecuentes.
CircuitBreakerFailureThreshold	5	3-10	Fallos antes de abrir. Bajar en prod para aislamiento rápido.
CircuitBreakerOpenSeconds	45	30-60	Tiempo abierto. Aumentar si CU inestable.
MaxRetries	3	1-5	Reintentos. 3 es standard, 5 si flakey network.
InitialRetryDelayMs	500	300-1000	Backoff inicial. 500ms es conservador.

Recomendaciones:

- **Local:** MaxConcurrentCalls: 2, CircuitBreakerFailureThreshold: 3 (debug rápido)
- **Staging:** MaxConcurrentCalls: 6, CircuitBreakerFailureThreshold: 5, MaxRetries: 3
- **Production:** MaxConcurrentCalls: 8-10, CircuitBreakerFailureThreshold: 3, MaxRetries: 5

Fallback & Cost:

- Si CU falla, ExtraerActivity cae a GPT (~2-3x más caro)
- Circuit breaker abierto → fallback automático a GPT
- Monitor: CU_CircuitBreaker_Open alerts en AppInsights

2.3.2 Azure OpenAI (GPT-4o-mini)

Configuración actual:

```
{
  "GptFallback": {
    "Enabled": true,
    "TimeoutSeconds": 30,
    "MaxTokens": 2000,
    "Temperature": 0.0,
    "DeploymentName": "gpt-4o-mini"
  },
  "Classification.GptFallback": {
    "TimeoutSeconds": 30,
    "MaxTokens": 150,
    "FallbackThreshold": 0.6
  }
}
```

Límites & Características:

- Token limit: 128K context (gpt-4o-mini)
- Rate limit: Depends on Azure deployment (típico: 100 req/min, 50K tokens/min)
- Timeout default: 30 sec
- Cost: ~\$0.01-0.05 per classification (150-500 tokens)

Tuning:

Parámetro	Default	Rango	Impacto
TimeoutSeconds	30	20-60	Timeout total. Aumentar si prompt largo o modelo lento.
MaxTokens	150 (classif), 2000 (extract)	100-3000	Limita output. Reducir si costo crítico.
Temperature	0.0	0.0-0.3	Creatividad. Mantener bajo (determinista).
FallbackThreshold	0.6 (classif)	0.5-0.8	Confianza mínima CU antes de fallback GPT. Bajar para más consistencia.

Recomendaciones:

- **Cost optimization:** MaxTokens 150→100, FallbackThreshold 0.6→0.7 (menos fallbacks)
- **Latency optimization:** TimeoutSeconds 30→20, MaxTokens 150→120
- **Reliability:** FallbackThreshold 0.6→0.5, TimeoutSeconds 30→40

Fallback chain:

1. Clasificación: rules → CU+DI → GPT
2. Extracción: CU → DI → GPT
3. Prompt: GPT (siempre)

2.3.3 Azure Document Intelligence (DI)

Uso: Classification + Extraction fallback + Markdown layout

Timeouts implícitos:

- Poll interval: 1000ms (appsettings)
- Total timeout: 120 sec (ClassificationPreparation o ExtraerActivity)

Tuning:

Parámetro	Actual	Recomendado	Notas
PollIntervalMs	1000	500-2000	Intervalo polling. Reducir para latencia, aumentar para carga.
TimeoutSeconds	120	90-180	Total. DI es generalmente rápido (< 30 sec).

2.3.4 DirectInvoice / Plugins REST/SOAP

Uso: IntegrarActivity para enriquecimiento de datos

Características:

- Connection pooling (default: 10)
- Timeout SOAP: típicamente 15 sec
- Network latency: +500ms (red corporativa)

Tuning:

Aspecto	Recomendación
Connection pool size	10-20 (aumentar si > 50 docs/sec)
Timeout individual	30-60 sec (según plugin)
Retry policy	Exponential backoff (1s, 2s, 4s)
Circuit breaker	Habilitar si plugin crítico pero inestable

2.4 Azure SQL Database

Actual: SQL Server 2022, EF Core 8, múltiples queries por doc (~5-10 queries)

Tuning points:

Parámetro	Default	Recomendado	Impacto
Connection pool size	100	100-150 (prod)	Conexiones simultáneas. Aumentar si > 100 concurrent docs.
Query timeout	30 sec	30-60 sec	Timeout por query. Aumentar para queries complejas.
DTU / vCore capacity	Depends on SKU	Scale per workload	Si DTU consistently > 80%, upgrade.
Read replicas	None	Considerar si read-heavy	Para scaling reads (analytics, deduplication).

Índices críticos:

- Ejecuciones.SHA256 (deduplication lookup): **MUST BE PRESENT**
- Ejecuciones.FechaEjecucion (histórico)
- ConfiguracionJson.Familia, Tipologia (resolución tipología)
- TipologiasConfiguradas.Código (lookup)

Recomendaciones:

- Verificar índices con: `SELECT name FROM sys.indexes WHERE object_id = OBJECT_ID('Ejecuciones');`
- Monitor DTU: `sys.resource_stats` en SSMS o Azure Portal
- Si DTU > 80% recurrente: Upgrade SKU o habilitar auto-scale (Serverless es opción)

2.5 Azure Storage (Blob)

Uso: Almacenamiento de PDFs (`documents/` container), deduplicación

Límites & Características:

- Account throughput: 20 Gbps ingress, 20 Gbps egress (Standard LRS)
- Blob IOPS: ~60K ops/sec (per storage account)
- Timeout default: 90 sec

Tuning:

Aspecto	Recomendación
Storage account SKU	Standard_GRS (geo-redundant) para prod
Partition strategy	Automatizado (date-based folders recomendado)
Connection pooling	Azure SDK maneja automáticamente
Timeout	90 sec default, aumentar a 120 sec si docs > 50MB
Blob tier	Hot (por defecto), considera Cool/Archive para > 30 días

Cost optimization:

- Enable lifecycle policy: Move to Cool after 30 days, Archive after 90 days
- Expected: ~\$0.001-0.005 per blob

2.6 Caching

Actual: Minimal in-app caching. Redis no habilitado.

Oportunidades de tuning:

Recurso	Hit rate esperado	Impacto	Recomendación
Tipología config (JSON)	95%+	-500ms latencia	In-app cache (60 min TTL)
Validación rules	95%+	-100ms latencia	In-app cache (60 min TTL)
Provider responses (CU extraction)	30-50% (duplicados)	-3 sec latencia	Redis (24h TTL) si duplicados frecuentes
Deduplication (SHA256 lookup)	5-10% (repeated)	-100ms latencia	DB index suficiente

Implementación recomendada:

- In-app cache:** IMemoryCache (tipología, validación rules) con 60-min TTL
- Distributed cache (Redis):** Si > 500 docs/day y deduplicación frecuente, agregar Redis para provider responses
- Blob storage:** Cachear bytes PDF en memoria solo si doc < 5MB

Cost estimado (Redis):

- Azure Cache for Redis (Basic, 1GB): ~\$35/month
- ROI: Break-even si > 500 duplicados/mes (~\$0.07 cada)

3. Paso-a-Paso: Identificar Cuellos de Botella

3.1 Health Check

Ejecutar esto antes de cualquier tuning:

```
# 1. Verificar estado de servicios
$services = @("AzureContentUnderstanding", "OpenAI", "SQL", "Storage", "Functions")
foreach ($svc in $services) {
    # Usar AppInsights query o Azure Portal status
    Write-Output "Checking $svc..."
}

# 2. Baseline latency (local o test)
# Ejecutar ingest de 1 documento simple, medir total time

# 3. Current throughput
# Monitor Functions metrics: Invocations, Execution time, Error count
```

Herramientas:

- Azure Portal → Function App → Monitor
- Application Insights → Performance → Custom metrics
- PowerShell: `Get-AzFunctionAppSetting`, `Get-AzWebAppMetrics`

3.2 Profiling

Identificar actividad lenta:

```
# Application Insights - KQL Query
customMetrics
| where name == "ActivityExecutionTime"
| extend ActivityName = tostring(customDimensions.ActivityName)
| summarize AvgTime = avg(value), MaxTime = max(value), P95Time = percentile(value, 95)
  by ActivityName
| order by AvgTime desc
```

Interpretar resultados:

- Si `ExtraerActivity` > 5 sec: CU lento o fallback a GPT frecuente
- Si `SubirGDCActivity` > 3 sec: SOAP timeout o network latency
- Si `ValidarActivity` > 1 sec: Reglas complejas, considerar regex optimization

3.3 Configuration Review

Checklist:

```
// host.json
{
  "maxConcurrentActivityFunctions": 4, // ← Aumentar si I/O-bound
  "maxConcurrentOrchestratorFunctions": 4, // ← Mantener bajo
  "extendedSessionsEnabled": false // ← true si latencia crítica
}

// appsettings.json - AzureContentUnderstanding
{
  "MaxConcurrentCalls": 4, // ← Aumentar a 8+ si throughput bajo
  "HardTimeoutSeconds": 90, // ← Aumentar a 120 si docs complejos
  "EnableCircuitBreaker": true, // ← Debe estar true
  "CircuitBreakerFailureThreshold": 5 // ← Bajar a 3 para aislamiento rápido
}

// appsettings.json - GptFallback
{
  "FallbackThreshold": 0.6, // ← Aumentar a 0.7+ para menos fallbacks
  "TimeoutSeconds": 30 // ← Verificar vs actual latency P95
}
```

3.4 External Dependencies

Verificar limites y health:

```
# 1. CU Rate limit
# Monitor: AppInsights "CU_RateLimitExceeded" events

# 2. OpenAI token usage
# Monitor: Tokens per minute, cost trend

# 3. SQL performance
# Query: SELECT * FROM sys.dm_exec_query_stats
# ORDER BY total_elapsed_time DESC

# 4. GDC SOAP availability
# Monitor: SubirGDCActivity error rate (target < 1%)

# 5. Network latency
# Ping: corbizondms.corpnet, serbwidd03.sareb.srb
# Expected: < 50ms
```

4. Recomendaciones de Configuración

4.1 Local / Dev Environment

host.json:

```
{
  "version": "2.0",
  "extensions": {
    "durableTask": {
      "hubName": "DocumentIAHub",
      "storageProvider": {
        "connectionStringName": "AzureWebJobsStorage"
      },
      "maxConcurrentActivityFunctions": 4,
      "maxConcurrentOrchestratorFunctions": 4,
      "extendedSessionsEnabled": false,
      "tracing": {
        "traceInputsAndOutputs": false
      }
    },
    "http": {
      "routePrefix": "api"
    }
  }
}
```

appsettings.Development.json (override):

```
{
  "Extraction": {
    "AzureContentUnderstanding": {
      "MaxConcurrentCalls": 2,
      "HardTimeoutSeconds": 60,
      "MaxRetries": 1
    }
  },
  "Classification": {
    "GptFallback": {
      "TimeoutSeconds": 20
    }
  },
  "HybridTdn": {
    "RescueTimeoutMs": 5000
  }
}
```

Rationale:

- Lower concurrency para debug más fácil
- Lower timeouts para fallos rápidos
- Fewer retries para ciclo de feedback rápido

4.2 Staging / Production Base

host.json:

```
{
  "version": "2.0",
  "extensions": {
    "durableTask": {
      "hubName": "DocumentIAHub",
      "storageProvider": {
        "connectionStringName": "AzureWebJobsStorage"
      },
      "maxConcurrentActivityFunctions": 8,
      "maxConcurrentOrchestratorFunctions": 4,
      "extendedSessionsEnabled": false,
      "tracing": {
        "traceInputsAndOutputs": false
      }
    }
  }
}
```

appsettings.Production.json (override):

```
{
  "Extraction": {
    "AzureContentUnderstanding": {
      "MaxConcurrentCalls": 8,
      "HardTimeoutSeconds": 90,
      "EnableCircuitBreaker": true,
      "CircuitBreakerFailureThreshold": 3,
      "CircuitBreakerOpenSeconds": 60,
      "MaxRetries": 5,
      "InitialRetryDelayMs": 500
    },
    "GptFallback": {
      "TimeoutSeconds": 60,
      "MaxTokens": 2000
    }
  },
  "Classification": {
    "GptFallback": {
      "TimeoutSeconds": 40,
      "MaxTokens": 150,
      "FallbackThreshold": 0.65
    }
  },
  "HybridTdn": {
    "RescueTimeoutMs": 10000,
    "MaxRetries": 2
  },
  "ClassificationPreparation": {
    "Enabled": true,
    "MaxPaginasClasificacionDefault": 3,
    "OverridesPorFamilia": {
      "sere": 5
    }
  }
}
```

Rationale:

- Higher concurrency para throughput
- Longer timeouts para variabilidad de red
- More retries para transitorios recovery
- Circuit breaker aggressive para aislamiento rápido

4.3 Production High-Throughput (1000+ docs/day)

Additional overrides:

```
{
  "extensions": {
    "durableTask": {
      "maxConcurrentActivityFunctions": 12,
      "extendedSessionsEnabled": true
    }
  },
  "Extraction": {
    "AzureContentUnderstanding": {
      "MaxConcurrentCalls": 12
    }
  }
}
```

Paired with:

- Azure Functions Premium Plan (dedicated compute)
- Auto-scale rules: Scale up if Execution time > 20 sec
- SQL: Upgrade to Premium tier (4 vCore min)
- Redis cache: Enabled para deduplication

5. Scaling Strategies

5.1 Vertical Scaling (Aumentar capacidad de instancia)

Recurso	De	A	Impacto	Costo
Function App	Consumption	Premium	+10-50% latency, +2-3x throughput	+50% monthly
Function App	Premium B1	Premium B2	+5-10% latency, +100% throughput	+2x monthly
SQL Database	Standard S0	Standard S1	+30% query capacity	+3x monthly
SQL Database	Standard → Premium	S2 → P1	+50-100% capacity	+10x monthly
Storage Account	Standard LRS	Standard GRS	+latency mínima, +redundancy	+1.5x monthly

Cuándo usar:

- Latency crítica pero throughput OK
- Transitorios (spike predecible, ej. fin de mes)
- Simplificar operaciones (1 instancia > 10)

5.2 Horizontal Scaling (Aumentar instancias)

Estrategia	Setup	Impacto	Complejidad
Auto-scale (Consumption)	CPU/Duration triggers	Automático, +latencia media	Media
Auto-scale (Premium)	Custom rules (AppInsights)	Controlado, predecible	Alta
Multiple instances (fixed)	10+ instances, load balancer	Máximo throughput, complejo	Muy alta
Durable Task Hub partitioning	Multiple hubs	Escalado infinito, debugging difícil	Muy alta

Recomendación:

- Staging/Prod < 50 docs/sec: Auto-scale Consumption (simple)
- Prod > 50 docs/sec: Auto-scale Premium con custom metrics (latency + error rate)
- Prod > 200 docs/sec: Multiple hubs + Traffic Manager (enterprise)

5.3 Queue-Based Async Processing

Problema: API latency crítica pero throughput bajo

Solución: Desacoplar ingesta de procesamiento con Azure Service Bus

Arquitectura:

```
HTTP API (sync) → Service Bus Queue → Function (async) → Result DB
                        ↓ (polling)
                  Client notificado
```

Implementación:

```
// Trigger
[Function("IngestDocumentAsync")]
public async Task<ActionResult> IngestDocumentAsync(
    [HttpTrigger(AuthorizationLevel.Function, "post", Route = "ingest-async")] HttpRequest req,
    [ServiceBusTrigger("documents-queue", ...)] IAsyncCollector<ContratoEntrada> queueClient)
{
    var entrada = JsonSerializer.Deserialize<ContratoEntrada>(await req.Body);
    await queueClient.AddAsync(entrada);
    return new AcceptedResult("/api/status/{correlationId}", entrada.Trazabilidad.CorrelationId);
}

// Processor
[Function("ProcessDocumentFromQueue")]
public async Task ProcessDocumentAsync(
    [ServiceBusTrigger("documents-queue", ...)] ContratoEntrada entrada)
{
    // Mismo orquestador como ahora
    var result = await _orchestrationClient.StartNewAsync("DocumentProcessOrchestrator", entrada);
}
```

Impacto:

- API latency: 100-200ms (vs 5-12 sec)
- Throughput: Unlimited (queue throttles)
- Cost: +\$0.75/month Service Bus

6. Optimizaciones por Escenario

6.1 High Throughput (1000+ docs/day)

Meta: Maximizar docs/sec, aceptar latencia moderada (< 30 sec P99)

Recomendaciones:

1. Infrastructure:

- Azure Functions Premium Plan (B2+)
- Auto-scale: min 5 instances, max 20+
- SQL: Premium tier (P2 mínimo)
- Redis cache: Enabled
- Service Bus: Standard o Premium (para async batching)

2. Configuration:

```
{
  "maxConcurrentActivityFunctions": 12-16,
  "extendedSessionsEnabled": true,
  "CU.MaxConcurrentCalls": 12,
  "GPT.TimeoutSeconds": 40,
  "BatchSize": 50 // Procesar en lotes
}
```

3. Code optimization:

- Parallelizar activities independientes (ex: Asset resolver + Validation)
- Implementar request batching para CU (si API soporta)
- Use connection pooling (SQL, HTTP)
- Enable compression (JSON, gzip)

4. Monitoring:

- Alert si Activity queue length > 100
- Alert si CU circuit breaker open
- Alert si SQL DTU > 90%

Expected results:

- Throughput: 1000+ docs/day
- Avg latency: 8-10 sec
- Cost per doc: \$0.03-0.04
- Infrastructure monthly: ~\$5K-8K

6.2 Low Latency (< 10 sec requirement)

Meta: Minimizar P99 latency, aceptar throughput menor

Recomendaciones:

1. Infrastructure:

- Azure Functions Premium Plan (B1+, co-located region)
- Fixed instances (5+, no auto-scale para predictabilidad)
- SQL: Premium tier (low latency storage)
- In-process cache (IMemoryCache)
- Connection pooling tuned high

2. Configuration:

```
{
  "extendedSessionsEnabled": true,
  "CU.MaxConcurrentCalls": 4,
  "CU.HardTimeoutSeconds": 60,
  "GPT.TimeoutSeconds": 30,
  "PromptDefaults.MaxTokens": 100 // Menos tokens = más rápido
}
```

3. Code optimization:

- Pre-warm connections (SQL, HTTP)
- Parallel classification attempts (rules + CU async, fallback GPT if CU slow)
- Skip optional activities (markdown, prompt) si `classificationOnly=true`
- Cache tipología config in memory

4. Pipeline optimization:

- Reduce classification flows (ex: rules only, no GPT fallback)
- Skip GDC upload si posible (`SkipGDCUpload=true`)

- Skip asset resolver si no necesario
- Use smaller PDF clips para classification (ClassificationPreparation)

Expected results:

- Avg latency: 4-6 sec
- P99 latency: < 10 sec
- Throughput: 50-100 docs/sec
- Cost per doc: \$0.04-0.06 (más caro por pre-warming)

Trade-offs:

- Throughput limitado por concurrency
- Cost alto (fixed instances, little utilization)
- Limited fallbacks (less reliability)

6.3 Cost Optimization

Meta: Minimizar costo por documento, aceptar latencia variable

Recomendaciones:

1. Infrastructure:

- Azure Functions Consumption Plan (pay-per-execution)
- SQL: Standard tier (S0-S1)
- No cache (costo adicional)
- Service Bus: Basic tier (o queue-based deferred processing)

2. Configuration:

```
{
  "maxConcurrentActivityFunctions": 4,
  "Classification.GptFallback.FallbackThreshold": 0.75, // Menos fallbacks
  "GPT.MaxTokens": 100,
  "Extraction.GptFallback.Enabled": false, // Solo CU/DI, sin GPT
  "ClassificationPreparation.MaxPaginasClasificacionDefault": 2 // Menos páginas
}
```

3. Provider tuning:

- Use cheaper provider primero: Rules → CU → (skip GPT or use GPT-3.5-turbo if available)
- Batch requests a CU (si API soporta)
- Use classification-only mode para high-volume batches
- Skip extraction si no necesario

4. Pipeline optimization:

- Skip validación si confianza > 0.95
- Skip integración (plugins) si campos necesarios ya extraídos
- Skip GDC upload, hacer batch después
- Use scheduled batch processing off-peak (cheaper compute)

Expected results:

- Cost per doc: \$0.01-0.02
- Throughput: 50-200 docs/day
- Avg latency: 5-15 sec (variable)
- Monthly infrastructure: ~\$500-1K

Trade-offs:

- Latency unpredictable (Consumption throttles)
- Limited fallbacks (cost cutting)

- Batch operations latency (deferred processing)

7. Monitoreo & Alertas

7.1 Métricas Clave

En Application Insights (custom metrics):

```
// 1. Latency percentiles
customMetrics
| where name == "DocumentProcessingLatency"
| summarize P50 = percentile(value, 50),
             P95 = percentile(value, 95),
             P99 = percentile(value, 99),
             Avg = avg(value)
| by bin(timestamp, 5m)
```

```
// 2. Throughput (docs/min)
customMetrics
| where name == "DocumentProcessed"
| summarize Count = dcount(customDimensions.DocumentId)
| by bin(timestamp, 1m)
```

```
// 3. Error rate
customMetrics
| where name == "DocumentProcessingError"
| summarize ErrorCount = dcount(customDimensions.DocumentId),
             TotalCount = dcount(customDimensions.DocumentId)
| extend ErrorRate = (ErrorCount * 100.0) / TotalCount
```

```
// 4. Cost per classification
customMetrics
| where name == "ProviderCost"
| extend Provider = tostring(customDimensions.Provider)
| summarize TotalCost = sum(value)
| by Provider
```

```
// 5. Activity duration distribution
customMetrics
| where name == "ActivityExecutionTime"
| extend ActivityName = tostring(customDimensions.ActivityName)
| summarize Duration = percentile(value, 95)
| by ActivityName
| order by Duration desc
```

7.2 Alertas Recomendadas

Alerta	Condición	Umbral	Acción
Latency spike	P99 latency > 30 sec	> 2 sec aumento en 5 min	Page on-call, check CU/GDC status
Error rate high	Error rate > 5%	> 2% aumento en 5 min	Investigate provider failures
CU circuit breaker	CircuitBreakerOpen = true	Any occurrence	CU inestable, escalate
Cost spike	Daily cost > baseline + 50%	\$150/day (adjust per environment)	Review plugin usage, check failures
Activity timeout	Activity timeouts > 1%	> 1 timeout per min	Increase timeout or parallelize
Throughput drop	Docs/min < baseline - 50%	< 3 docs/min (adjust)	Rate limiting or service degradation
SQL DTU high	DTU usage > 90%	> 5 min sustained	Upgrade SKU or optimize queries
GDC upload failure	SubirGDC error rate > 10%	> 1 error per min	GDC maintenance or network issue

Implementación (Application Insights):

```
{
  "alerts": [
    {
      "name": "DocumentProcessingLatency-P99-Alert",
      "metric": "DocumentProcessingLatency",
      "operator": "GreaterThan",
      "threshold": 30000, // ms
      "severity": 2,
      "window": 5, // min
      "frequency": 1 // min
    }
  ]
}
```

8. Benchmarks de Referencia

8.1 Expected Performance por Tipología

Tipología	Complejidad	Tiempo estimado	Providers	Notas
Nota Simple	Simple	2-4 sec	CU only	Sin extracción compleja
Tasación	Medium	5-8 sec	CU + GPT fallback	CU puede fallar en tasaciones
Escritura	Complex	8-15 sec	CU + DI + GPT, plugins	Múltiples reintentos típicos
Contrato	Very complex	12-25 sec	All providers	Markdown layout + prompts
Identificativo (DNI)	Very simple	1-2 sec	Rules only	No IA necesaria

8.2 Cost Baseline (USD)

Componente	Costo/doc	Variación	Factores
Azure CU	\$0.002-0.005	±50%	Páginas, fallbacks
OpenAI GPT	\$0.01-0.05	±200%	Prompt length, tokens response
Azure SQL	\$0.0001-0.001	±50%	# queries, DTU billing
Azure Storage	\$0.00001	negligible	Blob ops
Azure Functions	\$0.0002-0.001	±50%	Execution time, memory
Total	\$0.02-0.06	±100%	Mix de providers

Cost tracker (KQL):

```
customMetrics
| where name in ("CU_Cost", "GPT_Cost", "SQL_Cost")
| extend Provider = name
| summarize DailyTotal = sum(value)
| extend CostPerDoc = DailyTotal / 100 // Ajustar con real doc count
```

9. Troubleshooting Performance

9.1 Latency Spike

Síntomas:

- P99 latency aumentó de 20 sec a 40 sec (o más)
- Algunos documentos toman 1-2 min (usualmente < 20 sec)

Diagnóstico:

```
// 1. Identificar actividad lenta
customMetrics
| where name == "ActivityExecutionTime"
| where timestamp > ago(1h)
| extend ActivityName = tostring(customDimensions.ActivityName)
| summarize AvgTime = avg(value), MaxTime = max(value), P95Time = percentile(value, 95)
| by ActivityName
| order by MaxTime desc
| limit 5
```

```
// 2. Verificar si es provider-specific
exceptions
| where timestamp > ago(1h)
| where customDimensions.ActivityName in ("ExtraerActivity", "ClasificarActivity", "SubirGDCActivity")
| summarize Count = dcount(customDimensions.DocumentId)
| by customDimensions.ActivityName
```

Causas comunes:

Causa	Síntoma	Solución
CU rate limited	ExtraerActivity > 10 sec	Aumentar <code>MaxConcurrentCalls</code> , implement queue
GDC SOAP slow	SubirGDCActivity > 5 sec	Aumentar timeout a 180s, check network
SQL DTU high	VerificarDuplicado > 1 sec	Upgrade SKU, check long-running queries
OpenAI token limit	ClasificarActivity timeout	Reduce <code>MaxTokens</code> , use cheaper model
Network latency	All activities +500ms	Check corporative network, latency to Azure DC

Remediación rápida:

```
# 1. Aumentar activity timeout
$appsettings = Get-Content "appsettings.json" -AsJson
$appsettings.Classification.GptFallback.TimeoutSeconds = 40
$appsettings | ConvertTo-Json | Set-Content "appsettings.json"

# 2. Aumentar concurrency
$hostJson = Get-Content "host.json" -AsJson
$hostJson.extensions.durableTask.maxConcurrentActivityFunctions = 8
$hostJson | ConvertTo-Json | Set-Content "host.json"

# 3. Restart function app
Restart-AzFunctionApp -ResourceGroupName "rg-sareb-prod" -Name "srbappprodcai"
```

9.2 Throughput Drops

Síntomas:

- Documentos procesados por minuto cae de 10 a 2
- Queue de processing crece
- CPU/memory low

Diagnóstico:

```
// 1. Verificar retry storms
customMetrics
| where name == "ActivityRetry"
| summarize RetryCount = sum(value)
| by bin(timestamp, 5m)
| order by timestamp desc
```

```
// 2. Verificar circuit breakers
customMetrics
| where name == "CircuitBreakerOpen"
| where customDimensions.Provider == "AzureContentUnderstanding"
| summarize Count = count()
```

```
// 3. Verificar rate limiting
customMetrics
| where name == "RateLimitExceeded"
| summarize Count = count()
| by customDimensions.Provider
```

Causas comunes:

Causa	Síntoma	Solución
Retry storm	High activity retry rate	Bajar FallbackThreshold, increase initial timeout
Rate limit (CU)	Circuit breaker abierto	Aumentar MaxConcurrentCalls, implement queue
Rate limit (OpenAI)	GPT timeouts	Use token limit policy, reduce MaxTokens
SQL connection exhausted	Query timeouts	Increase pool size, check long-running queries
Function throttling	All activities queued	Upgrade to Premium plan, increase concurrency

Remediación rápida:

```
# 1. Reduce fallback threshold (menos reintentos)
$cfg = Get-Content "appsettings.json" | ConvertFrom-Json
$cfg.Classification.GptFallback.FallbackThreshold = 0.75
$cfg | ConvertTo-Json | Set-Content "appsettings.json"

# 2. Reset circuit breaker
# Esperar CircuitBreakerOpenSeconds (45 sec default)
# 0 reiniciar función app

# 3. Scale out
New-AzFunctionApp -ResourceGroupName "rg-sareb-prod" -Name "srbappprodcai2" -Plan "Premium"
```

9.3 High Cost

Síntomas:

- Costo diario aumentó de \$50 a \$200 (4x)
- Costo por documento > \$0.10

Diagnóstico:

```
// 1. Costo por provider
customMetrics
| where name == "ProviderCost"
| summarize TotalCost = sum(value)
| by customDimensions.Provider
| order by TotalCost desc
```

```
// 2. Costo por tipología
customMetrics
| where name == "ProviderCost"
| extend Tipologia = tostring(customDimensions.Tipologia)
| summarize TotalCost = sum(value)
| by Tipologia
| order by TotalCost desc
```

```
// 3. Fallback rate (GPT usage)
customMetrics
| where name == "ProviderFallback"
| extend FromProvider = tostring(customDimensions.FromProvider),
    ToProvider = tostring(customDimensions.ToProvider)
| summarize FallbackCount = count()
| by FromProvider, ToProvider
```

Causas comunes:

Causa	Síntoma	Solución
Excessive fallback to GPT	GPT cost > 50% total	Increase FallbackThreshold, improve CU model
Retries (failed calls)	High retry count + failed	Check provider health, increase timeout
Verbose prompts	GPT tokens high	Reduce MaxTokens, optimize prompt length
Expensive plugin	Plugin cost > CU cost	Use cheaper alternative, implement caching
Inefficient rules	High rule evaluation cost	Optimize regex, reduce rule complexity

Remediación rápida:

```
# 1. Increase fallback threshold (menos GPT)
$cfg = Get-Content "appsettings.json" | ConvertFrom-Json
$cfg.Classification.GptFallback.FallbackThreshold = 0.80
$cfg.Extraction.GptFallback.Enabled = $false # Disable GPT fallback for extraction
$cfg | ConvertTo-Json | Set-Content "appsettings.json"

# 2. Reduce token usage
$cfg.PromptDefaults.MaxTokens = 500 # Was 1600
$cfg.Classification.GptFallback.MaxTokens = 100 # Was 150
$cfg | ConvertTo-Json | Set-Content "appsettings.json"

# 3. Monitor change
# Comparar costo en 24h post-change
```

10. Checklist: Production Performance Readiness

Usar antes de producción:

- ☐ Load test ejecutado (1000 docs simulados)
- ☐ Latency P99 < 30 sec (or SLA específico)
- ☐ Error rate < 1% (excluye reintentos exitosos)
- ☐ Throughput meets baseline (ej. 100 docs/day)
- ☐ Cost per doc estimated & approved by finance
- ☐ Alertas configuradas & tested (al menos 3 alertas)
- ☐ Auto-scale policies definidas (min/max instances)
- ☐ Runbook de escalation creado (who to page, escalation chain)
- ☐ Performance baseline established (baseline latency, throughput, cost)
- ☐ Failover strategy tested (CU down → GPT fallback, GDC down → skip upload)
- ☐ SQL indexes verified (SHA256, Familia, Tipología)
- ☐ Cache strategy validated (if used)
- ☐ Monitoring dashboard created (custom metrics)
- ☐ Team trained on performance tuning
- ☐ Documentation updated (runbooks, procedures)

11. Referencias

Documentación oficial:

- [Azure Durable Functions - Host.json documentation](#)
- [Azure Functions performance tuning](#)
- [Azure Content Understanding - Rate limits](#)
- [Application Insights - Performance monitoring](#)

Documentación interna:

- `01_ARQUITECTURA_SISTEMA.md` - Architecture overview
- `03_DISEÑO_TECNICO_DETALLADO.md` - Pipeline & activities detail
- `05_MANUAL_USO_CONFIGURACION.md` - Configuration guide
- `docs/observabilidad/OBSERVABILIDAD_KQL.md` - KQL queries & monitoring

Herramientas útiles:

- Azure Portal → Application Insights → Performance
- Azure Portal → Function App → Monitor
- SSMS → Activity Monitor (SQL queries)

- PowerShell: `Get-AzFunctionAppMetrics` , `Get-AzWebAppSetting`